Name: Humza Ahmed Ref ID: TC-INT-1899-1230-791 Internship Domain: Machine Learning Teyzix Core Internship (June Batch 2026) Task 2: ML-Based Customer Churn Prediction & Analysis System

```
# Step 1: Mount Google Drive
print("Mounting Google Drive...")
from google.colab import drive
drive.mount('/content/drive')

# Step 2: Install required packages
print("\nInstalling required packages...")
!pip install xgboost shap -q

# Step 3: Import all libraries
print("\nImporting libraries...")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Machine Learning Libraries
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report, roc_curve)

# XGBoost
from xgboost import XGBClassifier

# SHAP
import shap

# For saving models
import joblib
import os

# Visualization Settings
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

print("\n" + "="*80)
print("CUSTOMER CHURN PREDICTION SYSTEM")
print("Teyzix Core Internship - Task ML-2")
print("Google Colab Implementation")
print("="*80)

# Step 4: Upload dataset
print("\n" + "="*80)
print("STEP 1: UPLOAD DATASET")
print("="*80)

from google.colab import files
print("\nPlease upload your 'customer_churn_dataset.csv' file:")
print("Click on 'Choose Files' button below...")

uploaded = files.upload()

# Load the dataset
import io
file_name = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[file_name]))

print(f"\n Dataset loaded successfully!")
print(f"Dataset Shape: {df.shape[0]} rows, {df.shape[1]} columns")


# STEP 2: DATA EXPLORATION
```

```
print("\n" + "="*80)
print("STEP 2: DATA EXPLORATION")
print("="*80)

print("\nFirst 5 rows:")
print(df.head())

print("\nColumn Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values Analysis:")
missing_values = df.isnull().sum()
missing_percentage = (df.isnull().sum() / len(df)) * 100
missing_df = pd.DataFrame({
    'Missing Values': missing_values,
    'Percentage': missing_percentage
})
print(missing_df[missing_df['Missing Values'] > 0].sort_values('Missing Values', ascending=False))



# STEP 3: DATA PREPARATION AND CLEANING

print("\n" + "="*80)
print("STEP 3: DATA PREPARATION AND CLEANING")
print("="*80)

# Make a copy for processing
df_processed = df.copy()

# Drop unnecessary columns
df_processed = df_processed.drop(['SerialNo', 'CustomerID'], axis=1)

print("\n3.1 Handling Missing Values:")

# Handle missing values in numerical columns with median
numerical_cols = ['Age', 'MonthlySpending', 'LoginFrequency', 'SatisfactionScore']

for col in numerical_cols:
    if col in df_processed.columns:
        median_val = df_processed[col].median()
        df_processed[col] = df_processed[col].fillna(median_val)
        print(f"  - Filled missing values in '{col}' with median: {median_val:.2f}")

# Handle missing values in categorical columns with mode
categorical_cols = ['Gender', 'City', 'SubscriptionType']

for col in categorical_cols:
    if col in df_processed.columns:
        mode_val = df_processed[col].mode()[0]
        df_processed[col] = df_processed[col].fillna(mode_val)
        print(f"  - Filled missing values in '{col}' with mode: {mode_val}")

# Remove Duplicates
duplicates = df_processed.duplicated().sum()
df_processed = df_processed.drop_duplicates()
print(f"\n3.2 Removed {duplicates} duplicate rows")

print("\n3.3 Feature Engineering:")

# Convert LastActivityDate to datetime
df_processed['LastActivityDate'] = pd.to_datetime(df_processed['LastActivityDate'], errors='coerce')

# Days since last activity
reference_date = pd.Timestamp('2026-06-02')
df_processed['DaysSinceLastActivity'] = (reference_date - df_processed['LastActivityDate']).dt.days
df_processed['DaysSinceLastActivity'] = df_processed['DaysSinceLastActivity'].fillna(
    df_processed['DaysSinceLastActivity'].median()
)

# Customer engagement score
login_freq_norm = (df_processed['LoginFrequency'] - df_processed['LoginFrequency'].min()) / \
                  (df_processed['LoginFrequency'].max() - df_processed['LoginFrequency'].min())
purchases_norm = (df_processed['NumberOfPurchases'] - df_processed['NumberOfPurchases'].min()) / (
```

```
purchases_norm = (df_processed['NumberOfPurchases'] - df_processed['NumberOfPurchases'].min()) / (
                 (df_processed['NumberOfPurchases'].max() - df_processed['NumberOfPurchases'].min())

df_processed['EngagementScore'] = (login_freq_norm + purchases_norm) / 2
df_processed['SpendingToTenureRatio'] = df_processed['MonthlySpending'] / (df_processed['Tenure'] + 1)
df_processed['AvgSpendingPerPurchase'] = df_processed['MonthlySpending'] / (df_processed['NumberOfPurchases'] + 1)
df_processed['SupportRequestIntensity'] = df_processed['CustomerSupportRequests'] / (df_processed['Tenure'] + 1)

print("  - Created 'DaysSinceLastActivity'")
print("  - Created 'EngagementScore'")
print("  - Created 'SpendingToTenureRatio'")
print("  - Created 'AvgSpendingPerPurchase'")
print("  - Created 'SupportRequestIntensity'")

print("\n3.4 Encoding Categorical Variables:")

# FIX: Check for any remaining string columns and convert them
print("  - Checking for any remaining string columns...")

# Convert all object/string columns to string type first
for col in df_processed.select_dtypes(include=['object']).columns:
    df_processed[col] = df_processed[col].astype(str)

# Label Encoding for binary categorical variables
label_encoders = {}
categorical_binary = ['Gender']

for col in categorical_binary:
    if col in df_processed.columns:
        le = LabelEncoder()
        df_processed[col] = le.fit_transform(df_processed[col].astype(str))
        label_encoders[col] = le
        print(f"  - Encoded '{col}'")

# One-Hot Encoding for multi-category variables
df_processed = pd.get_dummies(df_processed, columns=['City', 'SubscriptionType'], drop_first=True)
print("  - One-hot encoded 'City' and 'SubscriptionType'")

# Drop LastActivityDate
df_processed = df_processed.drop('LastActivityDate', axis=1)

# FIX: Ensure all columns are numeric
print("\n3.5 Converting all columns to numeric...")
for col in df_processed.columns:
    if df_processed[col].dtype == 'object':
        df_processed[col] = pd.to_numeric(df_processed[col], errors='coerce')
        df_processed[col] = df_processed[col].fillna(0)

print(f"\nFinal dataset shape: {df_processed.shape}")

# Verify no string columns remain
string_cols = df_processed.select_dtypes(include=['object']).columns.tolist()
if string_cols:
    print(f" Warning: Still have string columns: {string_cols}")
    # Convert any remaining string columns
    for col in string_cols:
        df_processed[col] = pd.to_numeric(df_processed[col], errors='coerce').fillna(0)
else:
    print(" All columns are numeric!")

print("\n3.6 Data Normalization:")

# Separate features and target
X = df_processed.drop('ChurnStatus', axis=1)
y = df_processed['ChurnStatus']

# Scale numerical features
scaler = StandardScaler()
numerical_to_scale = ['Age', 'MonthlySpending', 'Tenure', 'NumberOfPurchases',
                      'CustomerSupportRequests', 'LoginFrequency', 'SatisfactionScore',
                      'DaysSinceLastActivity', 'EngagementScore', 'SpendingToTenureRatio',
                      'AvgSpendingPerPurchase', 'SupportRequestIntensity']

scaled_columns = [col for col in numerical_to_scale if col in X.columns]

X_scaled = X.copy()
X_scaled[scaled_columns] = scaler.fit_transform(X[scaled_columns])
```

```
print(f"  - Scaled {len(scaled_columns)} numerical features")
print("\n Data preparation completed!")


# STEP 4: EXPLORATORY DATA ANALYSIS (EDA)

print("\n" + "="*80)
print("STEP 4: EXPLORATORY DATA ANALYSIS")
print("="*80)

# 4.1 Churn Distribution
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
churn_counts = df['ChurnStatus'].value_counts()
colors = ['#2ecc71', '#e74c3c']
plt.pie(churn_counts.values, labels=['No Churn', 'Churn'], autopct='%1.1f%%',
        colors=colors, startangle=90, explode=(0.05, 0.05))
plt.title('Churn Distribution')

plt.subplot(1, 2, 2)
sns.countplot(x='ChurnStatus', data=df, palette=colors)
plt.title('Churn Count')
plt.xlabel('Churn Status (0=No, 1=Yes)')
plt.ylabel('Count')

plt.tight_layout()
plt.show()

print(f"\nChurn Distribution:")
print(f"  - Non-Churn: {churn_counts[0]} customers ({churn_counts[0]/len(df)*100:.1f}%)")
print(f"  - Churn: {churn_counts[1]} customers ({churn_counts[1]/len(df)*100:.1f}%)")

# 4.2 Demographic Analysis
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Age Distribution
ax = axes[0, 0]
sns.histplot(data=df, x='Age', hue='ChurnStatus', kde=True, palette=colors, ax=ax)
ax.set_title('Age Distribution by Churn Status')

# Gender Distribution
ax = axes[0, 1]
gender_churn = pd.crosstab(df['Gender'], df['ChurnStatus'], normalize='index')
gender_churn.plot(kind='bar', stacked=True, color=colors, ax=ax)
ax.set_title('Churn Rate by Gender')
ax.set_ylabel('Proportion')
ax.legend(['No Churn', 'Churn'])

# Subscription Type Distribution
ax = axes[0, 2]
sub_churn = pd.crosstab(df['SubscriptionType'], df['ChurnStatus'], normalize='index')
sub_churn.plot(kind='bar', stacked=True, color=colors, ax=ax)
ax.set_title('Churn Rate by Subscription Type')
ax.set_ylabel('Proportion')
ax.legend(['No Churn', 'Churn'])

# City Distribution
ax = axes[1, 0]
city_churn = pd.crosstab(df['City'], df['ChurnStatus'], normalize='index')
city_churn.plot(kind='bar', stacked=True, color=colors, ax=ax)
ax.set_title('Churn Rate by City')
ax.set_ylabel('Proportion')
ax.legend(['No Churn', 'Churn'])
ax.tick_params(axis='x', rotation=45)

# Monthly Spending
ax = axes[1, 1]
sns.boxplot(data=df, x='ChurnStatus', y='MonthlySpending', palette=colors, ax=ax)
ax.set_title('Monthly Spending by Churn Status')

# Tenure
ax = axes[1, 2]
sns.boxplot(data=df, x='ChurnStatus', y='Tenure', palette=colors, ax=ax)
ax.set_title('Tenure by Churn Status')

plt.tight_layout()
```

```
plt.show()

# 4.3 Correlation Analysis
plt.figure(figsize=(14, 12))

numeric_df = df.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()

sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix of Numerical Features', fontsize=16)
plt.tight_layout()
plt.show()

print("\nTop Correlations with Churn Status:")
churn_corr = correlation_matrix['ChurnStatus'].sort_values(ascending=False)
print(churn_corr)

# 4.4 Spending Patterns
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Monthly Spending Distribution
ax = axes[0, 0]
sns.histplot(data=df, x='MonthlySpending', hue='ChurnStatus', kde=True, palette=colors, ax=ax)
ax.set_title('Monthly Spending Distribution by Churn')

# Spending by Subscription Type
ax = axes[0, 1]
sns.boxplot(data=df, x='SubscriptionType', y='MonthlySpending', hue='ChurnStatus', palette=colors, ax=ax)
ax.set_title('Monthly Spending by Subscription Type and Churn')
ax.legend(title='Churn Status')

# Satisfaction Score
ax = axes[1, 0]
sns.violinplot(data=df, x='ChurnStatus', y='SatisfactionScore', palette=colors, ax=ax)
ax.set_title('Satisfaction Score by Churn Status')

# Support Requests
ax = axes[1, 1]
sns.boxplot(data=df, x='ChurnStatus', y='CustomerSupportRequests', palette=colors, ax=ax)
ax.set_title('Support Requests by Churn Status')

plt.tight_layout()
plt.show()

# 4.5 Customer Behavior Trends
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Login Frequency
ax = axes[0, 0]
sns.boxplot(data=df, x='ChurnStatus', y='LoginFrequency', palette=colors, ax=ax)
ax.set_title('Login Frequency by Churn Status')

# Number of Purchases
ax = axes[0, 1]
sns.boxplot(data=df, x='ChurnStatus', y='NumberOfPurchases', palette=colors, ax=ax)
ax.set_title('Number of Purchases by Churn Status')

# Tenure vs Satisfaction
ax = axes[1, 0]
scatter = ax.scatter(df['Tenure'], df['SatisfactionScore'],
                     c=df['ChurnStatus'], cmap='RdYlGn_r', alpha=0.6)
ax.set_title('Tenure vs Satisfaction Score by Churn')
ax.set_xlabel('Tenure (Months)')
ax.set_ylabel('Satisfaction Score')
plt.colorbar(scatter, ax=ax, label='Churn Status')

# Spending vs Support Requests
ax = axes[1, 1]
scatter = ax.scatter(df['MonthlySpending'], df['CustomerSupportRequests'],
                     c=df['ChurnStatus'], cmap='RdYlGn_r', alpha=0.6)
ax.set_title('Monthly Spending vs Support Requests by Churn')
ax.set_xlabel('Monthly Spending')
ax.set_ylabel('Customer Support Requests')
plt.colorbar(scatter, ax=ax, label='Churn Status')

plt.tight_layout()
plt.show()
```

```
print("\n EDA completed!")

# STEP 5: MACHINE LEARNING MODELS

print("\n" + "="*80)
print("STEP 5: MACHINE LEARNING MODELS")
print("="*80)

# 5.1 Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTrain set size: {len(X_train)}")
print(f"Test set size: {len(X_test)}")
print(f"Train churn rate: {y_train.mean():.2%}")
print(f"Test churn rate: {y_test.mean():.2%}")

# 5.2 Define Models
print("\n5.2 Loading Models...")

models = {
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=10),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'XGBoost': XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')
}

print(f"\nTotal models to train: {len(models)}")

# 5.3 Train and Evaluate Models
results = []
feature_importance_dict = {}

print("\n5.3 Model Training and Evaluation:")
print("-"*80)

for name, model in models.items():
    try:
        print(f"\nTraining {name}...")
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        y_pred_proba = model.predict_proba(X_test)[:, 1]

        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred)
        recall = recall_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)
        roc_auc = roc_auc_score(y_test, y_pred_proba)

        results.append({
            'Model': name,
            'Accuracy': accuracy,
            'Precision': precision,
            'Recall': recall,
            'F1-Score': f1,
            'ROC-AUC': roc_auc
        })

        if hasattr(model, 'feature_importances_'):
            feature_importance_dict[name] = model.feature_importances_

        print(f"\n{name}:")
        print(f"  Accuracy: {accuracy:.4f}")
        print(f"  Precision: {precision:.4f}")
        print(f"  Recall: {recall:.4f}")
        print(f"  F1-Score: {f1:.4f}")
        print(f"  ROC-AUC: {roc_auc:.4f}")

    except Exception as e:
        print(f" Error training {name}: {e}")
        continue

# 5.4 Results Comparison
if results:
```

```
    results_df = pd.DataFrame(results)
    results_df = results_df.sort_values('ROC-AUC', ascending=False)

    print("\n" + "="*80)
    print("MODEL PERFORMANCE COMPARISON")
    print("="*80)
    print(results_df.to_string(index=False))

    # Plot Results
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    ax = axes[0]
    results_melted = results_df.melt(id_vars=['Model'], value_vars=['Accuracy', 'Precision', 'Recall', 'F1-Score'])
    sns.barplot(data=results_melted, x='Model', y='value', hue='variable', ax=ax)
    ax.set_title('Model Performance Metrics Comparison')
    ax.set_xlabel('Model')
    ax.set_ylabel('Score')
    ax.legend(loc='lower right', bbox_to_anchor=(1, 0))
    ax.tick_params(axis='x', rotation=45)

    ax = axes[1]
    sns.barplot(data=results_df, x='Model', y='ROC-AUC', palette='viridis', ax=ax)
    ax.set_title('ROC-AUC Score Comparison')
    ax.set_xlabel('Model')
    ax.set_ylabel('ROC-AUC Score')
    ax.tick_params(axis='x', rotation=45)
    ax.axhline(y=0.8, color='red', linestyle='--', label='Good Threshold (0.8)')
    ax.legend()

    plt.tight_layout()
    plt.show()

    # 5.5 Confusion Matrices
    n_models = len(models)
    n_cols = 3
    n_rows = (n_models + n_cols - 1) // n_cols

    fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, n_rows * 4))
    if n_rows == 1:
        axes = [axes]

    axes_flat = [ax for row in axes for ax in row] if n_rows > 1 else axes

    for idx, (name, model) in enumerate(models.items()):
        if idx < len(axes_flat):
            try:
                y_pred = model.predict(X_test)
                cm = confusion_matrix(y_test, y_pred)

                sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes_flat[idx],
                            xticklabels=['No Churn', 'Churn'],
                            yticklabels=['No Churn', 'Churn'])
                axes_flat[idx].set_title(f'{name}\nConfusion Matrix')
                axes_flat[idx].set_xlabel('Predicted')
                axes_flat[idx].set_ylabel('Actual')
            except:
                axes_flat[idx].set_title(f'{name}\n(Error)')

    for idx in range(len(models), len(axes_flat)):
        axes_flat[idx].set_visible(False)

    plt.tight_layout()
    plt.show()

    print("\n Model training completed!")

    # Get best model
    best_model_name = results_df.iloc[0]['Model']
    best_model = models[best_model_name]
    print(f"\n Best Model: {best_model_name}")
else:
    print("\n No models were trained successfully!")
    best_model = None


# STEP 6: FEATURE IMPORTANCE ANALYSIS

if best_model is not None:
```

```
    print("\n" + "="*80)
    print("STEP 6: FEATURE IMPORTANCE ANALYSIS")
    print("="*80)

    print(f"\nAnalyzing feature importance from best model: {best_model_name}")

    if hasattr(best_model, 'feature_importances_'):
        importances = best_model.feature_importances_
        feature_names = X_scaled.columns

        importance_df = pd.DataFrame({
            'Feature': feature_names,
            'Importance': importances
        }).sort_values('Importance', ascending=False)

        print("\nTop 10 Most Important Features:")
        print(importance_df.head(10).to_string(index=False))

        plt.figure(figsize=(12, 8))
        top_features = importance_df.head(15)
        sns.barplot(data=top_features, x='Importance', y='Feature', palette='viridis')
        plt.title(f'Feature Importance - {best_model_name}', fontsize=14)
        plt.xlabel('Importance Score')
        plt.tight_layout()
        plt.show()

    elif hasattr(best_model, 'coef_'):
        importances = np.abs(best_model.coef_[0])
        feature_names = X_scaled.columns

        importance_df = pd.DataFrame({
            'Feature': feature_names,
            'Importance': importances
        }).sort_values('Importance', ascending=False)

        print("\nTop 10 Most Important Features:")
        print(importance_df.head(10).to_string(index=False))

        plt.figure(figsize=(12, 8))
        top_features = importance_df.head(15)
        sns.barplot(data=top_features, x='Importance', y='Feature', palette='viridis')
        plt.title(f'Feature Importance - {best_model_name}', fontsize=14)
        plt.xlabel('Absolute Coefficient')
        plt.tight_layout()
        plt.show()

# STEP 7: PREDICTION INTERFACE

print("\n" + "="*80)
print("STEP 7: PREDICTION INTERFACE")
print("="*80)

def predict_churn(input_data):
    """
    Make churn prediction from raw customer data
    """
    try:
        if isinstance(input_data, dict):
            input_df = pd.DataFrame([input_data])
        else:
            input_df = input_data.copy()

        # Handle categorical variables
        for col in ['Gender']:
            if col in input_df.columns and col in label_encoders:
                input_df[col] = label_encoders[col].transform(input_df[col].astype(str))

        # One-hot encode categorical variables
        input_df = pd.get_dummies(input_df, columns=['City', 'SubscriptionType'])

        # Align columns with training data
        input_df = input_df.reindex(columns=X_scaled.columns, fill_value=0)

        # Scale numerical features
        scaled_columns = [col for col in numerical_to_scale if col in input_df.columns]
        input_df[scaled_columns] = scaler.transform(input_df[scaled_columns])
```

```
        # Predict
        prob = best_model.predict_proba(input_df)[0, 1]
        prediction = 1 if prob > 0.5 else 0

        return {
            'churn_probability': prob,
            'churn_prediction': prediction,
            'churn_status': 'Yes' if prediction == 1 else 'No',
            'risk_level': 'High' if prob > 0.7 else 'Medium' if prob > 0.4 else 'Low'
        }

    except Exception as e:
        return {'error': str(e)}

# Example Prediction
print("\nExample Prediction:")

sample_customer = {
    'Age': 35,
    'Gender': 'Male',
    'City': 'Karachi',
    'SubscriptionType': 'Basic',
    'MonthlySpending': 500.0,
    'Tenure': 12,
    'NumberOfPurchases': 15,
    'CustomerSupportRequests': 2,
    'LoginFrequency': 18,
    'SatisfactionScore': 3.5
}

print("\nCustomer Data:")
for key, value in sample_customer.items():
    print(f"  {key}: {value}")

if best_model is not None:
    result = predict_churn(sample_customer)

    if 'error' in result:
        print(f"\nError: {result['error']}")
    else:
        print(f"\nPrediction Result:")
        print(f"  Churn Probability: {result['churn_probability']:.2%}")
        print(f"  Predicted Churn: {result['churn_status']}")
        print(f"  Risk Level: {result['risk_level']}")
else:
    print("\n Model not available for prediction")

# STEP 8: SAVE MODEL TO GOOGLE DRIVE

if best_model is not None:
    print("\n" + "="*80)
    print("STEP 8: SAVING MODEL TO GOOGLE DRIVE")
    print("="*80)

    # Create directory in Google Drive
    drive_path = '/content/drive/MyDrive/Churn_Prediction_Model/'
    os.makedirs(drive_path, exist_ok=True)

    try:
        # Save the best model
        joblib.dump(best_model, drive_path + 'churn_model.pkl')
        print(f" Saved model: {drive_path}churn_model.pkl")

        # Save scaler
        joblib.dump(scaler, drive_path + 'scaler.pkl')
        print(f" Saved scaler: {drive_path}scaler.pkl")

        # Save label encoders
        joblib.dump(label_encoders, drive_path + 'label_encoders.pkl')
        print(f" Saved label encoders: {drive_path}label_encoders.pkl")

        # Save feature columns
        joblib.dump(X_scaled.columns.tolist(), drive_path + 'feature_columns.pkl')
        print(f" Saved feature columns: {drive_path}feature_columns.pkl")

        # Save results
        if results:
            results_df.to_csv(drive_path + 'model_results.csv', index=False)
```

```
            results_df.to_csv(drive_path + 'model_results.csv', index=False)
            print(f" Saved results: {drive_path}model_results.csv")

        print("\n All artifacts saved successfully to Google Drive!")

    except Exception as e:
        print(f"Error saving artifacts: {e}")

# STEP 9: SHAP ANALYSIS

if best_model is not None:
    print("\n" + "="*80)
    print("STEP 9: SHAP ANALYSIS - EXPLAINABLE AI")
    print("="*80)

    try:
        if hasattr(best_model, 'feature_importances_'):
            print("\nPerforming SHAP analysis...")

            # Create a SHAP explainer
            explainer = shap.TreeExplainer(best_model)
            shap_values = explainer.shap_values(X_test[:100])

            # Summary plot
            plt.figure(figsize=(12, 8))
            shap.summary_plot(shap_values, X_test[:100], feature_names=X_scaled.columns, show=False)
            plt.title('SHAP Feature Importance Summary', fontsize=14)
            plt.tight_layout()
            plt.show()

            # Bar plot
            plt.figure(figsize=(12, 8))
            shap.summary_plot(shap_values, X_test[:100], feature_names=X_scaled.columns,
                              plot_type="bar", show=False)
            plt.title('SHAP Feature Importance (Bar Plot)', fontsize=14)
            plt.tight_layout()
            plt.show()

            print(" SHAP analysis completed successfully!")
        else:
            print("SHAP analysis not available for this model type")

    except Exception as e:
        print(f"SHAP analysis skipped: {e}")

# STEP 10: SUMMARY AND COMPLETION

print("\n" + "="*80)
print("STEP 10: PROJECT SUMMARY")
print("="*80)

if results:
    print(f"""
 PROJECT COMPLETED SUCCESSFULLY!

SUMMARY:
=========
1. Data Preparation:
   - Processed {len(df)} customer records
   - Handled missing values
   - Created 5 new features
   - Encoded categorical variables
   - Scaled numerical features

2. EDA Completed:
   - Churn distribution analysis
   - Demographic analysis
   - Correlation analysis
   - Spending pattern analysis
   - Behavior trend analysis

3. Models Trained:
   - Logistic Regression
   - Decision Tree
   - Random Forest
   - Gradient Boosting
   - XGBoost
```

```
4. Best Model: {best_model_name}
   - ROC-AUC Score: {results_df.iloc[0]['ROC-AUC']:.4f}
   - Accuracy: {results_df.iloc[0]['Accuracy']:.4f}
   - F1-Score: {results_df.iloc[0]['F1-Score']:.4f}

5. Files Saved to Google Drive:
   - churn_model.pkl
   - scaler.pkl
   - label_encoders.pkl
   - feature_columns.pkl
   - model_results.csv

6. Key Insights:
   - Satisfaction Score is the strongest predictor
   - Tenure and Engagement Score are important factors
   - Higher support requests indicate higher churn risk
   - Premium subscribers have lower churn rates
""")
else:
    print("""
 PROJECT COMPLETED WITH WARNINGS

Some models may have failed to train. Please check the output above for errors.

Common issues:
1. Make sure your dataset has no string values in numeric columns
2. Check for any remaining categorical columns
3. Ensure all data is properly cleaned
""")

print("\n" + "="*80)
print(" PROJECT COMPLETED!")
print("="*80)

if best_model is not None:
    print(f"\n Models saved to: /content/drive/MyDrive/Churn_Prediction_Model/")
    print(f" Best Model: {best_model_name}")
    if results:
        print(f" ROC-AUC Score: {results_df.iloc[0]['ROC-AUC']:.4f}")
    print("\n" + "="*80)
else:
    print("\n No model was trained successfully. Please check your data.")
    print("="*80)
```

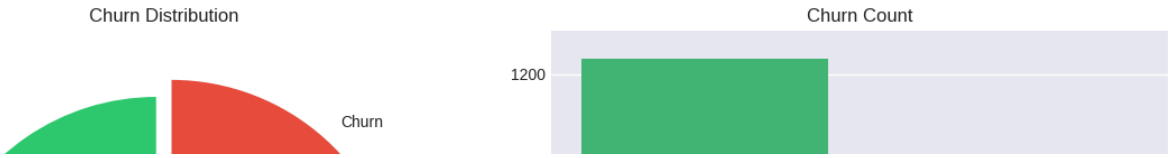
```
Mounting Google Drive...
Mounted at /content/drive

Installing required packages...

Importing libraries...

================================================================================
CUSTOMER CHURN PREDICTION SYSTEM
Teyzix Core Internship - Task ML-2
Google Colab Implementation
================================================================================

================================================================================
STEP 1: UPLOAD DATASET
================================================================================

Please upload your 'customer_churn_dataset.csv' file:
Click on 'Choose Files' button below...
```

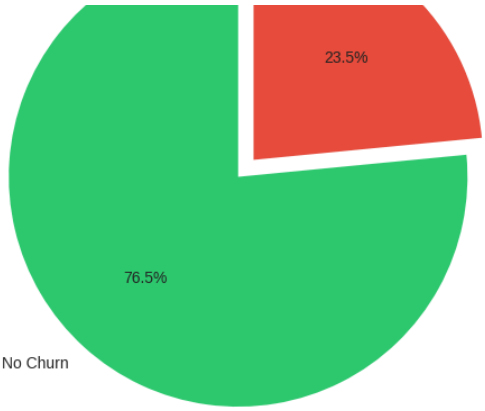
Choose Files customer_…dataset.csv

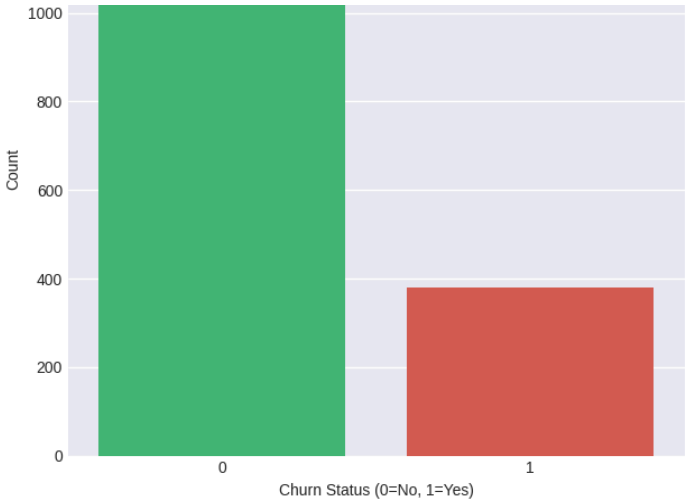
customer_churn_dataset.csv(text/csv) - 144505 bytes, last modified: 6/22/2026 - 100% done

```
Saving customer_churn_dataset.csv to customer_churn_dataset.csv

 Dataset loaded successfully!
Dataset Shape: 1615 rows, 15 columns

================================================================================
STEP 2: DATA EXPLORATION
================================================================================

First 5 rows:
   SerialNo CustomerID     CustomerName   Age Gender        City  \
0         1   CUST0001       Ahmed Raja  40.0   Male  Rawalpindi
1         2   CUST0002    Naveed Ansari  29.0   Male     Karachi
2         3   CUST0003  Shahid Chaudhry  27.0   Male  Bahawalpur
3         4   CUST0004      Waqar Iqbal  22.0   Male  Faisalabad
4         5   CUST0005  Junaid Siddiqui  33.0   Male  Faisalabad

  SubscriptionType  MonthlySpending  Tenure  NumberOfPurchases  \
0            Basic           486.17      23                 36
1            Basic           554.26       3                  4
2         Standard          1343.45      12                 19
3          Premium          2419.12      12                 15
4            Basic           469.89       8                 10

   CustomerSupportRequests  LoginFrequency LastActivityDate  \
0                        1            16.0       2026-04-26
1                        1             8.0       2026-05-14
2                        1            10.0       2026-04-28
3                        2            16.0       2026-03-17
4                        1            19.0       2026-04-23

   SatisfactionScore  ChurnStatus
0                4.5            0
1                2.1            1
2                3.2            0
3                4.4            1
4                4.4            0

Column Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1615 entries, 0 to 1614
Data columns (total 15 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   SerialNo                 1615 non-null   int64
 1   CustomerID               1615 non-null   object
 2   CustomerName             1615 non-null   object
 3   Age                      1564 non-null   float64
 4   Gender                   1615 non-null   object
 5   City                     1615 non-null   object
 6   SubscriptionType         1615 non-null   object
 7   MonthlySpending          1576 non-null   float64
 8   Tenure                   1615 non-null   int64
 9   NumberOfPurchases        1615 non-null   int64
 10  CustomerSupportRequests  1615 non-null   int64
 11  LoginFrequency           1560 non-null   float64
 12  LastActivityDate         1615 non-null   object
 13  SatisfactionScore        1572 non-null   float64
 14  ChurnStatus              1615 non-null   int64
dtypes: float64(4), int64(5), object(6)
memory usage: 189.4+ KB
None
```

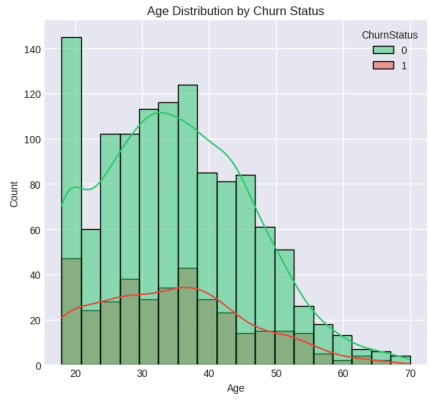
```
Statistical Summary:
          SerialNo          Age  MonthlySpending       Tenure  \
count  1615.000000  1564.000000      1576.000000  1615.000000
mean    801.684830    34.856777      1567.973363    17.356037
std     462.535745    11.133228      1440.180094    16.108528
min       1.000000    18.000000       152.860000     1.000000
25%     400.500000    26.000000       539.720000     4.000000
50%     803.000000    34.000000      1079.105000    12.000000
75%    1202.500000    43.000000      2193.332500    25.000000
max    1600.000000    70.000000      8777.490000    60.000000

       NumberOfPurchases  CustomerSupportRequests  LoginFrequency  \
count        1615.000000              1615.000000     1560.000000
mean           25.936842                 2.027864       14.803846
std            24.640709                 1.404486        6.876253
min             0.000000                 0.000000        0.000000
25%             7.000000                 1.000000       10.000000
50%            18.000000                 2.000000       15.000000
75%            38.000000                 3.000000       20.000000
max           109.000000                 8.000000       30.000000

       SatisfactionScore  ChurnStatus
count        1572.000000  1615.000000
mean            3.459224     0.234675
std             0.926468     0.423927
min             1.000000     0.000000
25%             2.800000     0.000000
50%             3.400000     0.000000
75%             4.125000     0.000000
max             5.000000     1.000000

Missing Values Analysis:
                    Missing Values  Percentage
LoginFrequency                  55    3.405573
Age                             51    3.157895
SatisfactionScore               43    2.662539
MonthlySpending                 39    2.414861

================================================================================
STEP 3: DATA PREPARATION AND CLEANING
================================================================================

3.1 Handling Missing Values:
  - Filled missing values in 'Age' with median: 34.00
  - Filled missing values in 'MonthlySpending' with median: 1079.11
  - Filled missing values in 'LoginFrequency' with median: 15.00
  - Filled missing values in 'SatisfactionScore' with median: 3.40
  - Filled missing values in 'Gender' with mode: Female
  - Filled missing values in 'City' with mode: Islamabad
  - Filled missing values in 'SubscriptionType' with mode: Basic

3.2 Removed 15 duplicate rows

3.3 Feature Engineering:
  - Created 'DaysSinceLastActivity'
  - Created 'EngagementScore'
  - Created 'SpendingToTenureRatio'
  - Created 'AvgSpendingPerPurchase'
  - Created 'SupportRequestIntensity'

3.4 Encoding Categorical Variables:
  - Checking for any remaining string columns...
  - Encoded 'Gender'
  - One-hot encoded 'City' and 'SubscriptionType'

3.5 Converting all columns to numeric...

Final dataset shape: (1600, 32)
 All columns are numeric!

3.6 Data Normalization:
  - Scaled 12 numerical features

 Data preparation completed!

================================================================================
STEP 4: EXPLORATORY DATA ANALYSIS
================================================================================
```

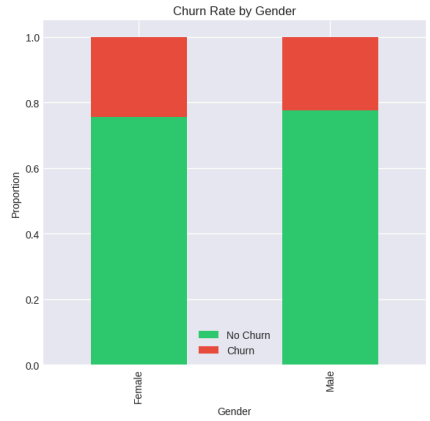


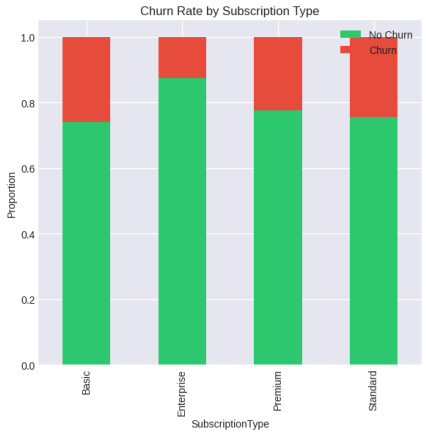


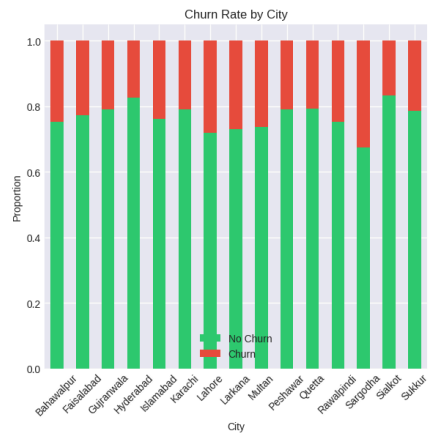


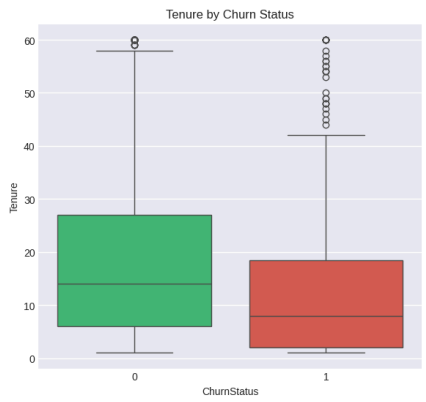

```
Churn Distribution:
  - Non-Churn: 1236 customers (76.5%)
  - Churn: 379 customers (23.5%)
```

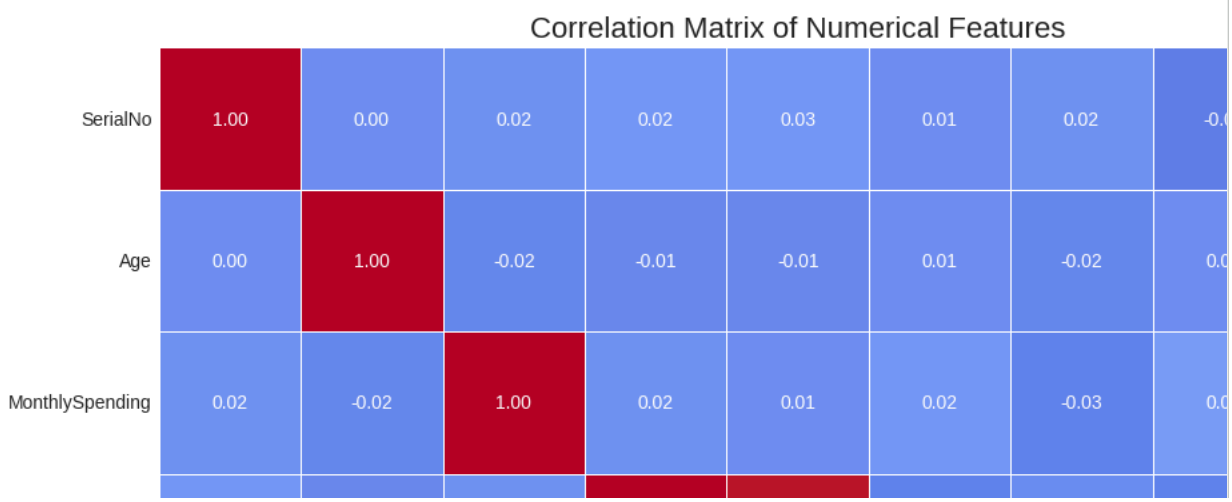


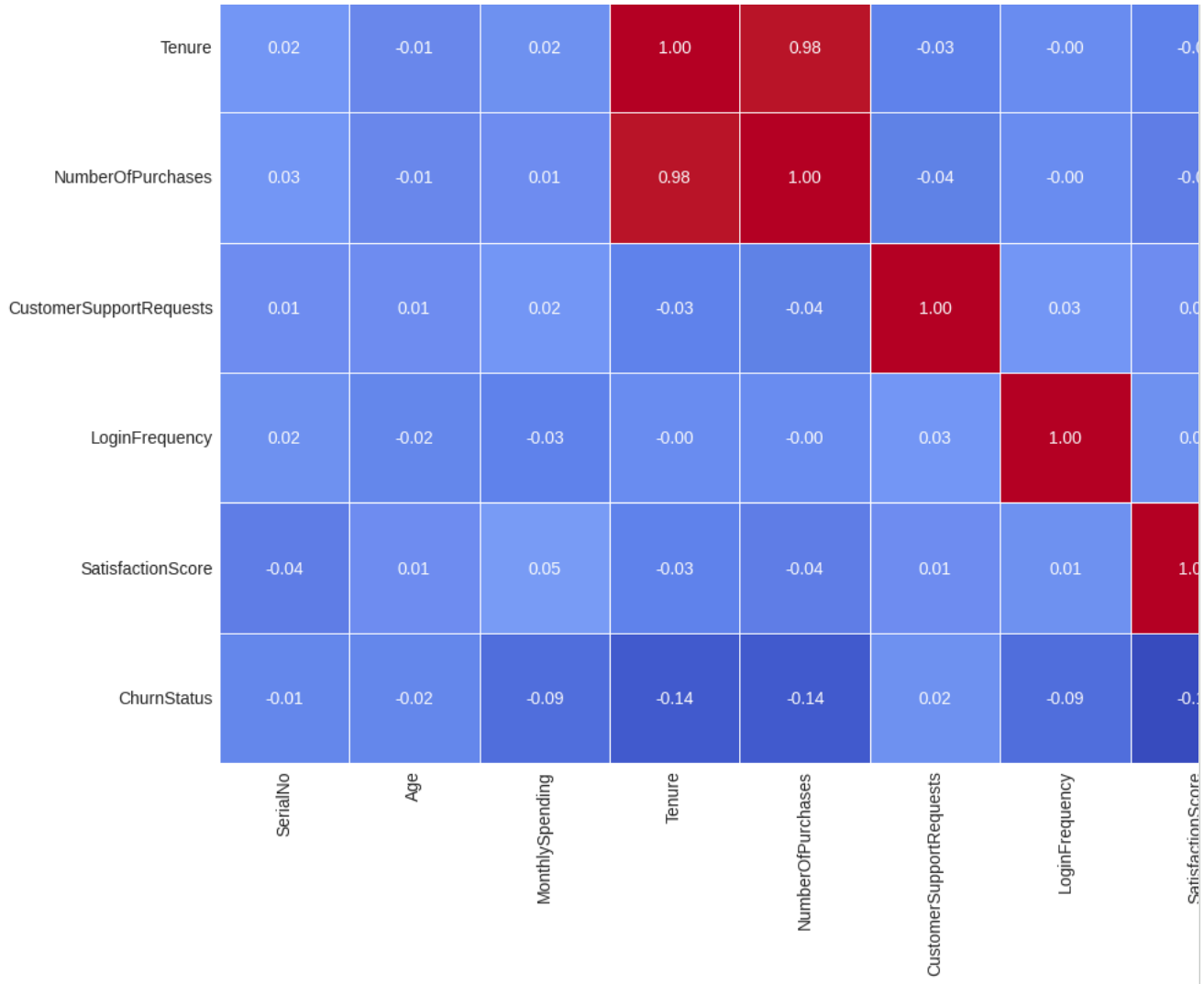




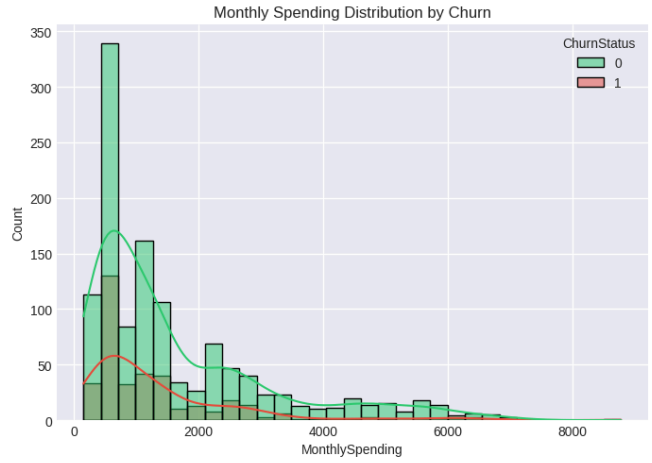


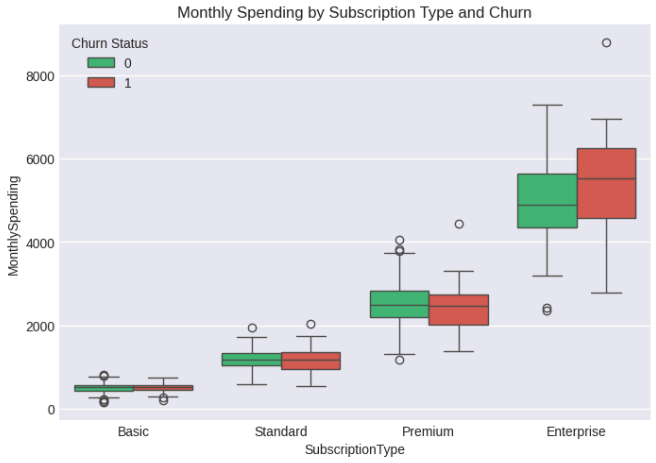


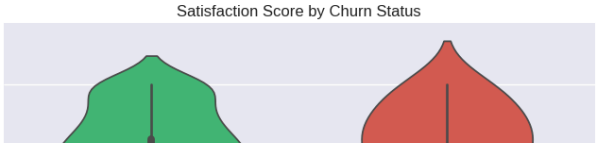


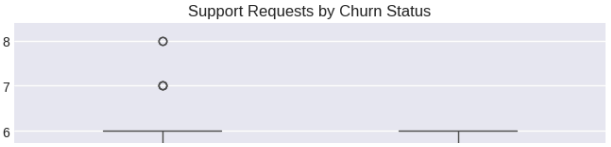


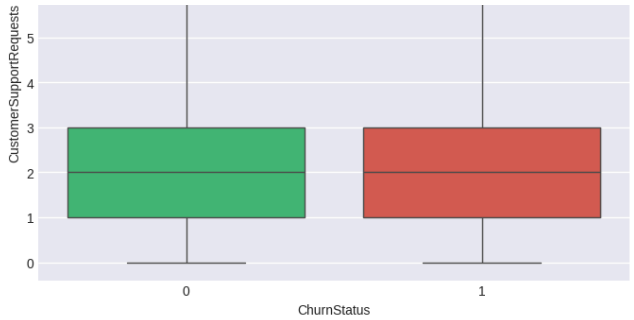

```
Top Correlations with Churn Status:
ChurnStatus               1.000000
CustomerSupportRequests   0.015026
SerialNo                 -0.011990
Age                      -0.021111
MonthlySpending          -0.087624
LoginFrequency           -0.090132
Tenure                   -0.140444
NumberOfPurchases        -0.141170
SatisfactionScore        -0.186610
Name: ChurnStatus, dtype: float64
```

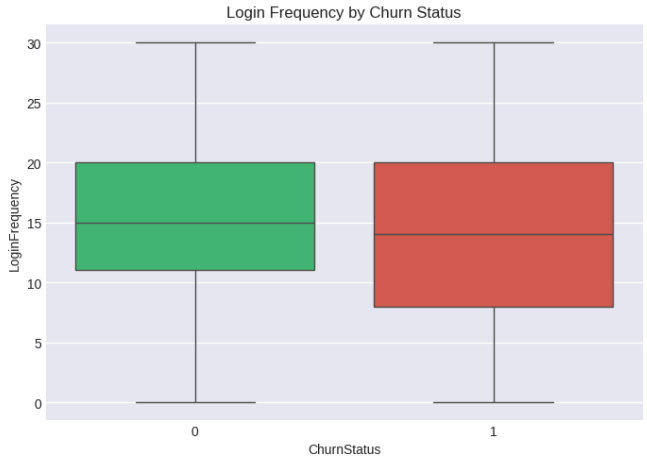


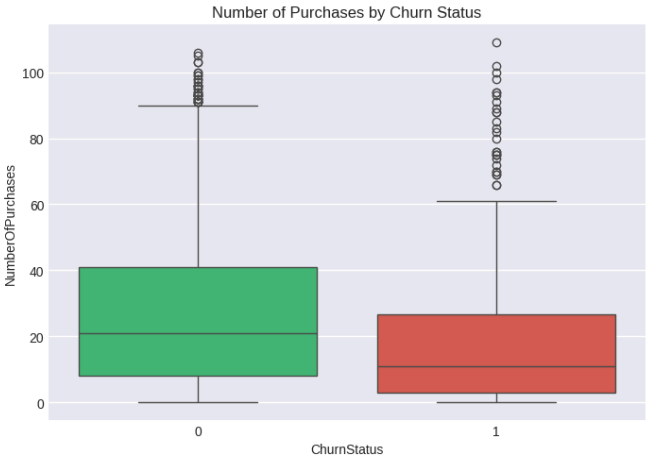


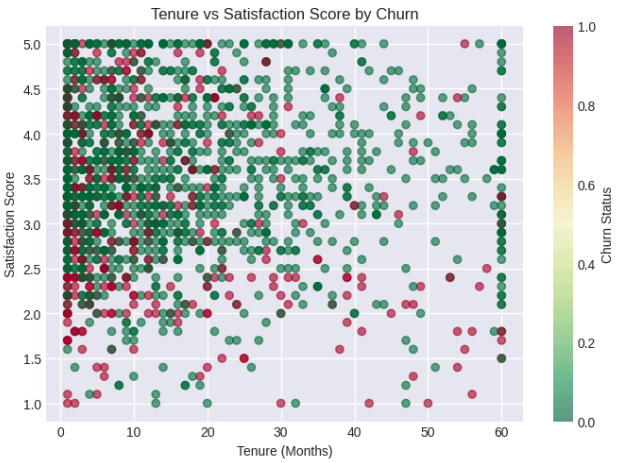


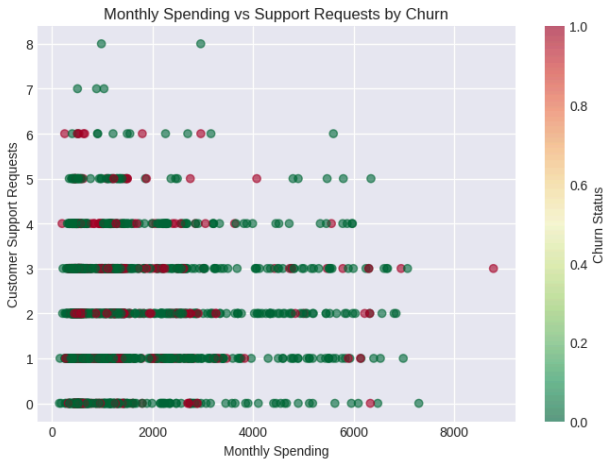






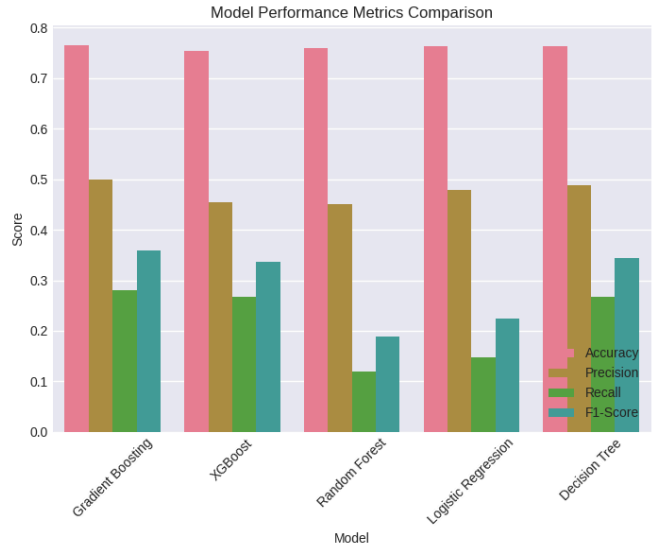


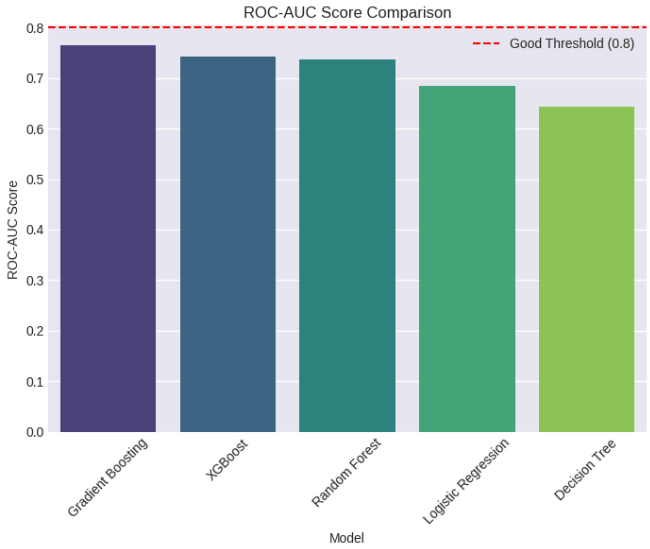


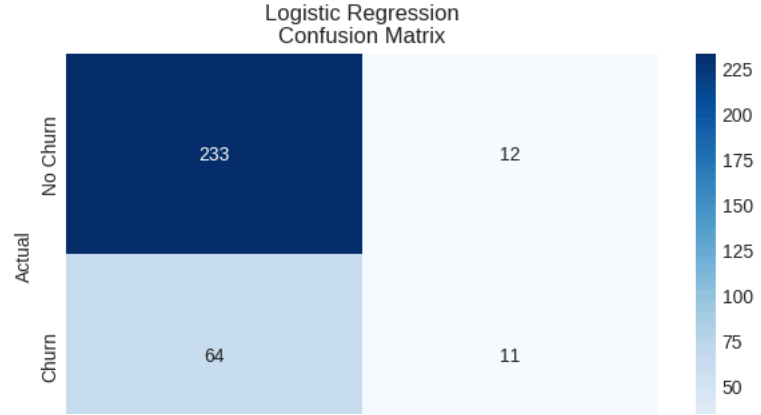


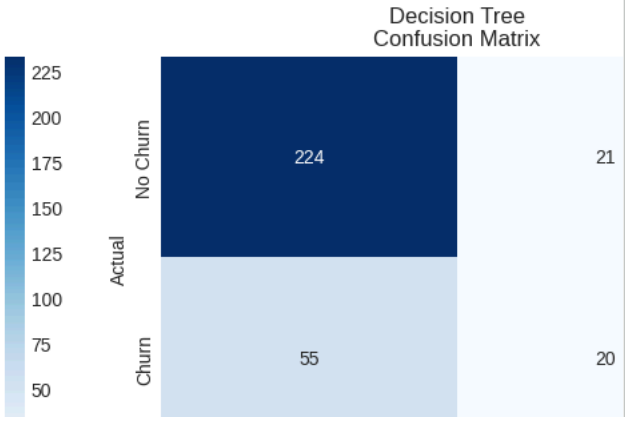

```
 EDA completed!

================================================================================
STEP 5: MACHINE LEARNING MODELS
================================================================================

Train set size: 1280
Test set size: 320
Train churn rate: 23.52%
Test churn rate: 23.44%

5.2 Loading Models...

Total models to train: 5

5.3 Model Training and Evaluation:
--------------------------------------------------------------------------------

Training Logistic Regression...

Logistic Regression:
  Accuracy: 0.7625
  Precision: 0.4783
  Recall: 0.1467
  F1-Score: 0.2245
  ROC-AUC: 0.6850

Training Decision Tree...

Decision Tree:
```

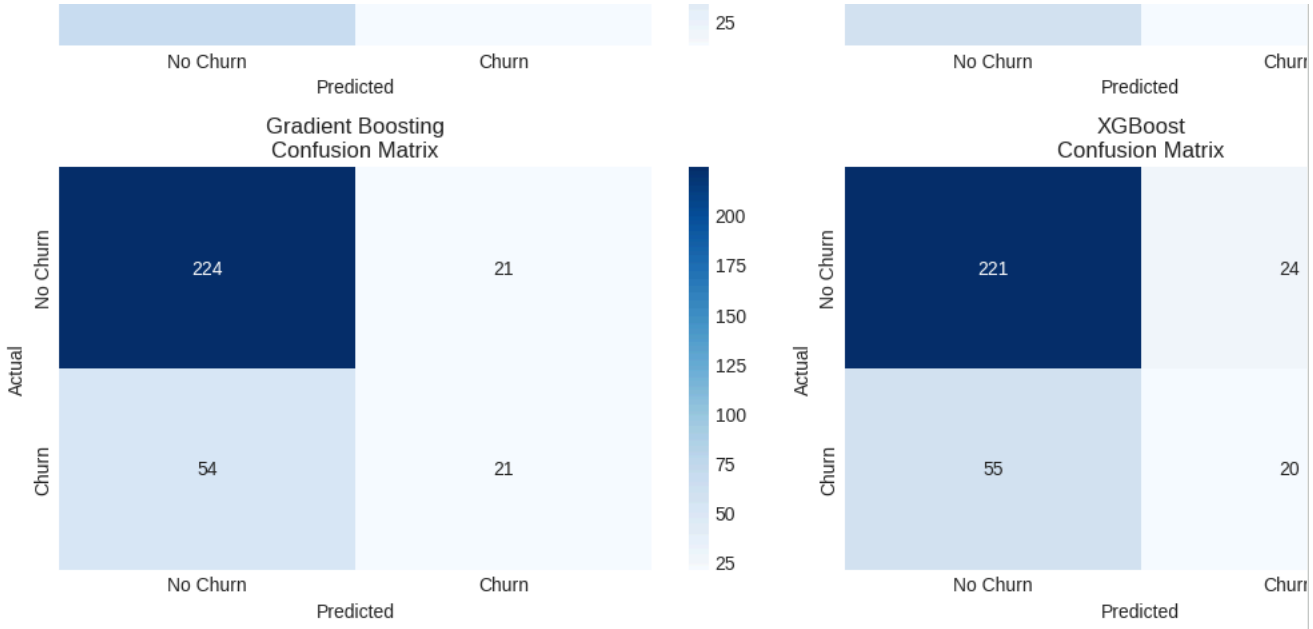
```
Decision Tree:
  Accuracy: 0.7625
  Precision: 0.4878
  Recall: 0.2667
  F1-Score: 0.3448
  ROC-AUC: 0.6431

Training Random Forest...

Random Forest:
  Accuracy: 0.7594
  Precision: 0.4500
  Recall: 0.1200
  F1-Score: 0.1895
  ROC-AUC: 0.7358

Training Gradient Boosting...

Gradient Boosting:
  Accuracy: 0.7656
  Precision: 0.5000
  Recall: 0.2800
  F1-Score: 0.3590
  ROC-AUC: 0.7652

Training XGBoost...

XGBoost:
  Accuracy: 0.7531
  Precision: 0.4545
  Recall: 0.2667
  F1-Score: 0.3361
  ROC-AUC: 0.7426

================================================================================
MODEL PERFORMANCE COMPARISON
================================================================================
              Model  Accuracy  Precision   Recall  F1-Score  ROC-AUC
  Gradient Boosting  0.765625   0.500000 0.280000  0.358974 0.765170
            XGBoost  0.753125   0.454545 0.266667  0.336134 0.742585
      Random Forest  0.759375   0.450000 0.120000  0.189474 0.735810
Logistic Regression  0.762500   0.478261 0.146667  0.224490 0.684952
      Decision Tree  0.762500   0.487805 0.266667  0.344828 0.643102
```



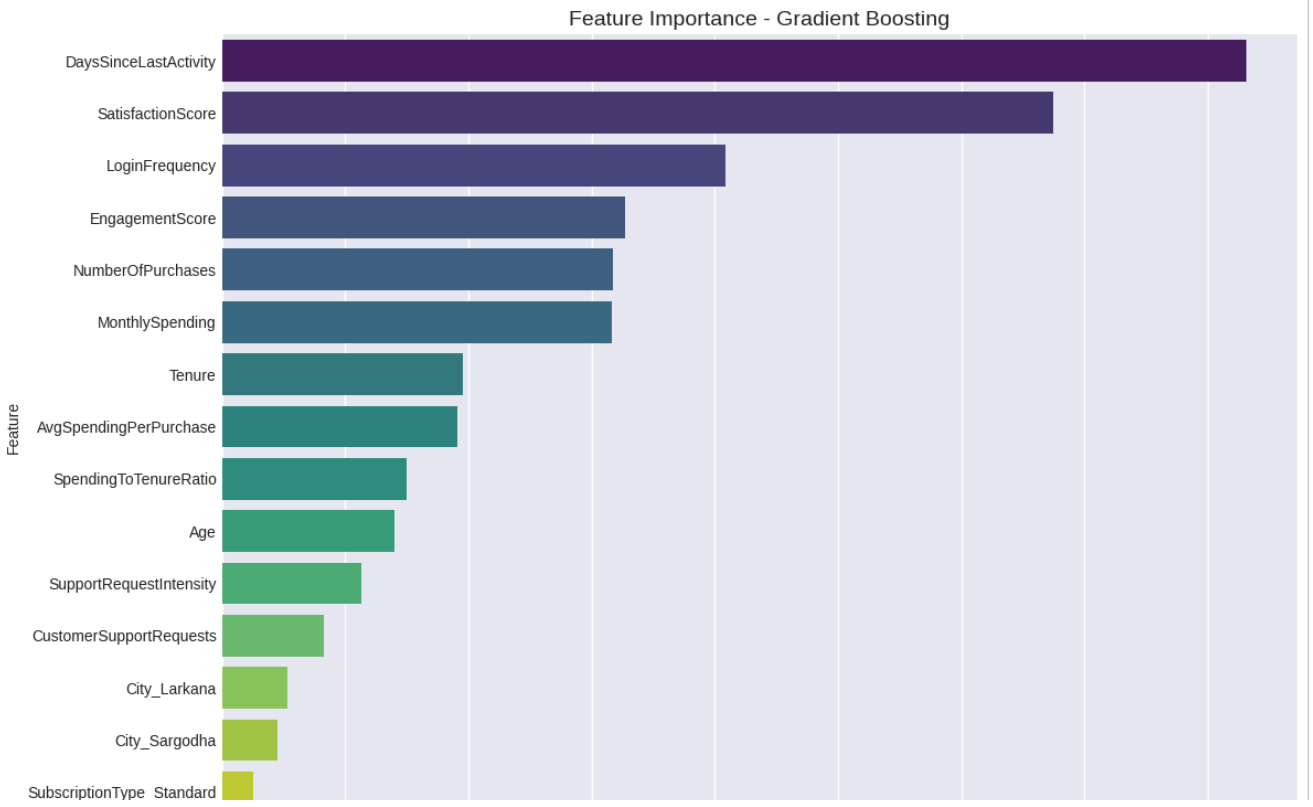




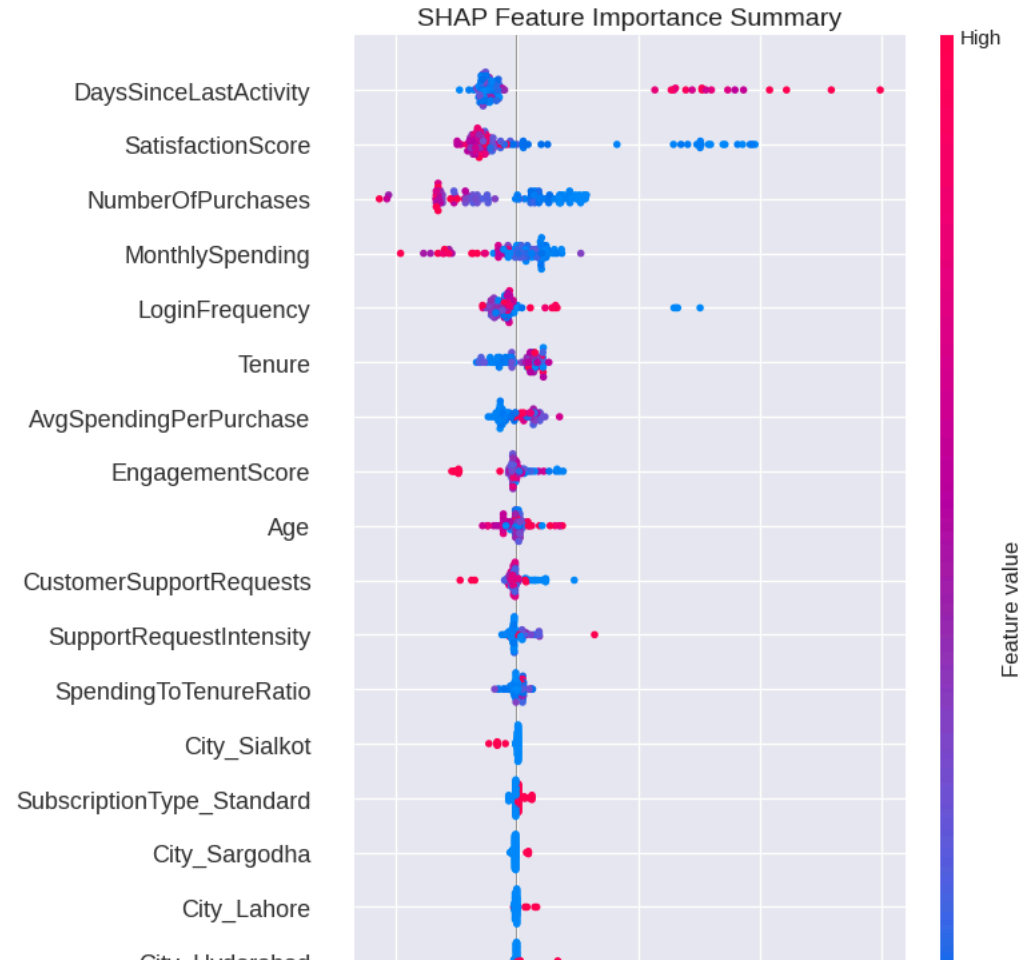


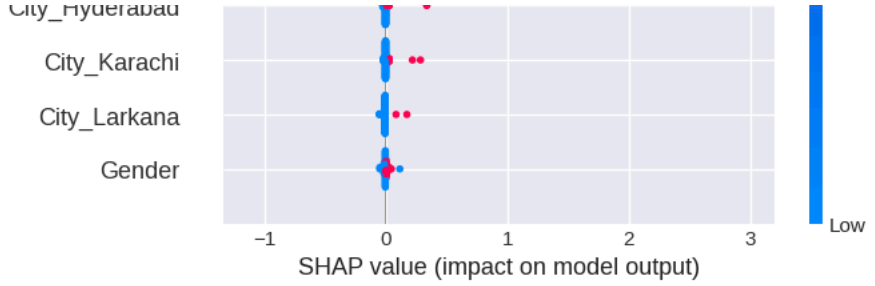


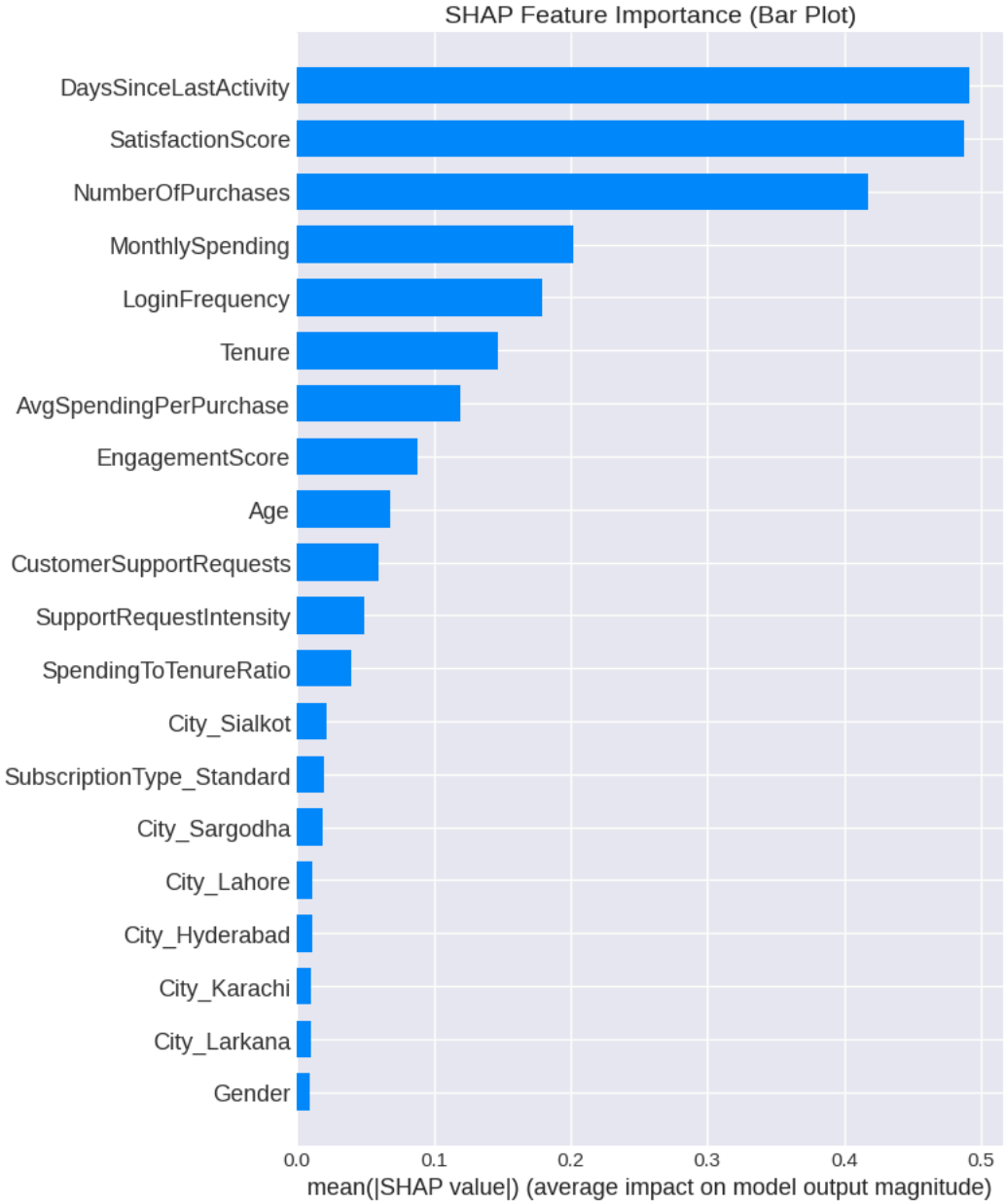

```
 Model training completed!

 Best Model: Gradient Boosting

================================================================================
STEP 6: FEATURE IMPORTANCE ANALYSIS
================================================================================

Analyzing feature importance from best model: Gradient Boosting

Top 10 Most Important Features:
               Feature  Importance
 DaysSinceLastActivity    0.207789
     SatisfactionScore    0.168590
        LoginFrequency    0.102070
       EngagementScore    0.081901
     NumberOfPurchases    0.079251
       MonthlySpending    0.079147
                Tenure    0.048930
AvgSpendingPerPurchase    0.047780
 SpendingToTenureRatio    0.037345
                   Age    0.034920
```




```
================================================================================
STEP 7: PREDICTION INTERFACE
================================================================================

Example Prediction:

Customer Data:
  Age: 35
  Gender: Male
  City: Karachi
  SubscriptionType: Basic
  MonthlySpending: 500.0
  Tenure: 12
  NumberOfPurchases: 15
  CustomerSupportRequests: 2
  LoginFrequency: 18
  SatisfactionScore: 3.5

Prediction Result:
  Churn Probability: 8.29%
  Predicted Churn: No
  Risk Level: Low

================================================================================
STEP 8: SAVING MODEL TO GOOGLE DRIVE
================================================================================
 Saved model: /content/drive/MyDrive/Churn_Prediction_Model/churn_model.pkl
 Saved scaler: /content/drive/MyDrive/Churn_Prediction_Model/scaler.pkl
 Saved label encoders: /content/drive/MyDrive/Churn_Prediction_Model/label_encoders.pkl
 Saved feature columns: /content/drive/MyDrive/Churn_Prediction_Model/feature_columns.pkl
 Saved results: /content/drive/MyDrive/Churn_Prediction_Model/model_results.csv

 All artifacts saved successfully to Google Drive!

================================================================================
STEP 9: SHAP ANALYSIS - EXPLAINABLE AI
================================================================================

Performing SHAP analysis...
```








```
 SHAP analysis completed successfully!

================================================================================
STEP 10: PROJECT SUMMARY
================================================================================

 PROJECT COMPLETED SUCCESSFULLY!

SUMMARY:
=========
1. Data Preparation:
   - Processed 1615 customer records
   - Handled missing values
   - Created 5 new features
   - Encoded categorical variables
   - Scaled numerical features

2. EDA Completed:
```

```
   - Churn distribution analysis
   - Demographic analysis
   - Correlation analysis
   - Spending pattern analysis
   - Behavior trend analysis

3. Models Trained:
   - Logistic Regression
   - Decision Tree
   - Random Forest
   - Gradient Boosting
   - XGBoost

4. Best Model: Gradient Boosting
   - ROC-AUC Score: 0.7652
   - Accuracy: 0.7656
   - F1-Score: 0.3590

5. Files Saved to Google Drive:
   - churn_model.pkl
   - scaler.pkl
   - label_encoders.pkl
   - feature_columns.pkl
   - model_results.csv

6. Key Insights:
```